

✓ Lezione di Ripasso Pre-Esame (4 ore)

Java Base → OOP → Collezioni → JDBC → Spring Boot

📌 **Obiettivo:** consolidare i concetti fondamentali e verificare la preparazione con esercizi + teoria.

STRUTTURA DELLA LEZIONE (4 ORE)

1) Warm-up + Java base (30 min)

Ripasso rapido

- Tipi primitivi e wrapper (`int` vs `Integer`)
- Variabili, scope, costanti (`final`)
- Operatori e strutture di controllo (`if`, `switch`, `for`, `while`)
- Array e stringhe (`String` immutabile, `StringBuilder`)

Mini esercizio (5-10 min)

Scrivere un programma che:

- legge 5 numeri
- stampa massimo, minimo e media

Domande teoriche flash

- Differenza tra `==` e `.equals()`
 - Perché `String` è immutabile?
 - Cosa sono i wrapper?
-

2) OOP completa (60 min)

Ripasso concetti fondamentali

- Classi e oggetti
- Costruttori e overload
- Incapsulamento (`private`, getter/setter)
- `static` e differenza tra membri statici e non statici
- Ereditarietà e `super`
- Override e overload
- Polimorfismo
- Classi astratte e interfacce
- `final` su variabili, metodi, classi

Esercitazione guidata (parte forte)

Progettare un mini-sistema:

Gestione Scuola

- `Persona` (astratta): nome, cognome
- `Studente`: classe frequentata
- `Docente`: materia insegnata
- metodo `stampaScheda()`

👉 usare polimorfismo con un array/lista di `Persona`.

Domande teoriche (stile esame)

- Differenza tra classe astratta e interfaccia
- Cos'è il polimorfismo? Esempio concreto
- Cosa significa override?
- Perché si usa incapsulamento?

3) Collezioni + Eccezioni + Generics (45 min)

Ripasso

- `List`, `Set`, `Map`
- differenza tra `ArrayList` e `LinkedList`
- differenza tra `HashSet` e `TreeSet`
- differenza tra `HashMap` e `TreeMap`
- iteratori
- `Comparable` e `Comparator`
- generics (`List<Studente>`)
- eccezioni (`try/catch`, checked vs unchecked)

Esercitazione pratica

Gestire una lista studenti:

- aggiunta studenti
- ricerca per cognome
- ordinamento per media voto
- stampa studenti promossi (≥ 6)

Domande teoriche

- Perché `HashSet` non ammette duplicati?
- Differenza tra checked e unchecked exceptions
- A cosa serve `Comparator`?

4) File + JSON/CSV (30 min)

Ripasso

- Lettura/scrittura file con `BufferedReader`, `FileWriter`
- Serializzazione concettuale
- Formato CSV
- cenni a JSON (struttura e uso tipico nelle API REST)

Mini esercizio

Salvare studenti su file CSV e ricaricarli.

Domande teoriche:

- differenza tra file di testo e file binario
- vantaggi del CSV
- quando usare JSON

5) JDBC + DAO Pattern (35 min)

Ripasso

- Connessione JDBC (`DriverManager`, `Connection`)
- `PreparedStatement` vs `Statement`
- SQL base: SELECT, INSERT, UPDATE, DELETE
- DAO Pattern (Data Access Object)
- gestione eccezioni SQL

Esercitazione (simulazione esame)

Creare:

- tabella `studenti(id, nome, cognome, media)`
- DAO con metodi:
 - `insert()`
 - `findAll()`
 - `findById()`
 - `delete()`

Domande teoriche:

- Perché `PreparedStatement` è migliore?
- Cos'è il DAO?
- cos'è la SQL Injection?

6) Spring Boot (40 min)

Ripasso concetti chiave

- cos'è Spring Boot e cosa semplifica

- struttura progetto (`controller`, `service`, `repository`)
- Dependency Injection e `@Autowired`
- annotazioni principali:
 - `@RestController`
 - `@GetMapping`, `@PostMapping`
 - `@Service`, `@Repository`
 - `@Entity`, `@Id`, `@GeneratedValue`
- Spring Data JPA: `JpaRepository`
- DTO e Mapper (perché servono)

Mini esercizio rapido

Creare API REST:

- `GET /studenti`
- `GET /studenti/{id}`
- `POST /studenti`
- `DELETE /studenti/{id}`

Domande teoriche:

- differenza tra Controller e Service
- cos'è la Dependency Injection?
- perché usare DTO invece dell'Entity direttamente?



Verifica Finale (20 min)

Simulazione domande da esame (orale + pratica)

Domande possibili:

1. Spiega incapsulamento e fai un esempio.
 2. Differenza tra override e overload.
 3. HashMap vs TreeMap.
 4. Checked vs unchecked exception.
 5. PreparedStatement: perché si usa?
 6. Cos'è un DAO?
 7. Cos'è Spring Boot?
 8. Cos'è una REST API?
 9. Controller-Service-Repository: responsabilità.
 10. DTO: perché è utile?
-



Consegna finale per casa (facoltativa)

Progetto completo mini:

- gestione studenti (CRUD)
 - salvataggio su DB
 - esposizione REST API con Spring Boot
-



Materiale consigliato per la lezione

- Schema OOP (UML semplice)
- Esempi di codice già pronti da completare
- DB già predisposto (MySQL o H2)
- Postman per test API